ContextKey - Book with confidence. Pay for the agreement you approved.

Problem statement

Camera and audiovisual rentals involve more than a price. Customers also agree on equipment, accessories, pickup and return dates, deposits, and cancellation conditions.

When these details are spread across quotations and messages, customers and businesses must manually check that the final booking reflects the agreement. A booking can show the correct total while containing a missing accessory, a different return time, or a changed refund condition.

ContextKey helps both parties identify and resolve these differences before payment.

What the application does

ContextKey turns a quotation and relevant messages into a concise agreement summary, with each condition linked to its source. The customer reviews and confirms that summary.

The application then compares the merchant's final booking against the confirmed agreement. It identifies matching conditions, meaningful differences, and points that need clarification. Equivalent wording should pass; missing information should remain unresolved.

Customers can also preview a cancellation scenario to understand its financial consequences. The merchant can correct the booking, or the customer can explicitly approve a revision.

Once the agreement is approved, the customer completes PayPal sandbox checkout. ContextKey records the approved agreement version alongside the payment identifiers and transaction status.

Example

A quotation includes a camera, two batteries, and a microphone for $200. The final booking still costs $200 but lists only one battery and excludes the microphone.

ContextKey highlights the missing items and shows the quotation that included them. The merchant corrects the booking, the customer approves it, and payment proceeds.

Value

Customers gain a clearer understanding of what they are buying. Rental businesses gain a structured way to confirm negotiated details before collecting payment.

The prototype demonstrates agreement verification and payment control. It does not claim that customer demand or dispute reduction has already been validated.

The hackathon requires meaningful integration of **PayPal sandbox and AI**. Sponsor tools are optional; PayPal must be central to the project. Devpost

Component	Use in ContextKey
PayPal sandbox	Run the complete checkout using test buyer and merchant accounts.
PayPal Checkout	Let the customer approve payment through PayPal.
Orders API v2	Create an order, retrieve its details, and capture payment after the application checks agreement approval.
`custom_id`	Include an internal booking/agreement reference for reconciliation. Store the full agreement in PostgreSQL.
`PayPal-Request-Id`	Make supported payment requests safe to retry without unintended duplicate operations.
PayPal webhooks	Receive payment events and reconcile transaction status on the backend.
APIMatic Context Plugin — optional	Help the coding assistant implement the PayPal integration using relevant API context.

PayPal documents order creation and capture, reconciliation through `custom_id`, request idempotency, and checkout webhooks. These provide the payment foundation; **your backend implements the agreement approval checks**. PayPal does not interpret or enforce your rental terms. PayPal Developer

APIMatic's plugin is a development aid, rather than the AI that compares agreements. Use it only if it helps, and document its actual contribution. Devpost

How AI and application code work together

AI handles	Application code handles
Extract conditions from text and PDFs	Validate structured fields
Compare wording and meaning	Calculate amounts and deadlines
Identify ambiguity and conflicting evidence	Enforce approval and agreement versions
Explain differences with source references	Create orders, capture payments, and reconcile status

The important payment rule is: **if the booking changes after approval, the customer must review it again before your backend permits capture.** Build this as a deliberate state transition, with immutable agreement versions.

Use React + Python + FastAPI + PostgreSQL for ContextKey. This is a good fit for document processing, AI integration, and PayPal APIs. Spring Boot is unnecessary.

Layer	Technology	Purpose
Frontend	React + TypeScript + Tailwind CSS	Customer review, merchant corrections, checkout, receipt
Backend	Python + FastAPI	Agreement processing, approvals, payment endpoints
Validation	Pydantic	Validate extracted conditions and API requests
Database	PostgreSQL + SQLAlchemy	Store bookings, evidence, agreement versions, payment records
Database migrations	Alembic	Manage database schema changes
Document extraction	PyMuPDF	Extract text from uploaded PDFs
AI	One model supporting structured outputs	Extract conditions, compare meaning, explain differences
Calculations	Python `Decimal` + `datetime`	Calculate money and cancellation deadlines
Payments	PayPal JavaScript SDK + Orders API v2	Buyer approval and backend order creation/capture
API requests	HTTPX	Call PayPal and other external APIs
Local setup	Docker Compose	Run the backend and PostgreSQL consistently